

ClassCord Server — Service de tchat multi-clients

Python 3.10+ · systemd · Docker · Sécurité · Interopérabilité SLAM — Semaine intensive

1. Contexte et objectifs

Projet de semaine intensive mené par les étudiants SISR : déploiement, sécurisation, supervision et industrialisation du serveur de tchat **ClassCord** (*server_classcord.py* — Python 3.10+), avec interopérabilité avec les clients Java développés par les étudiants SLAM. Objectif : mettre à disposition un service de communication interne fiable, documenté et exploitable.

2. Stack technique

Domaine	Choix techniques
OS	Linux Debian/Ubuntu — VM VirtualBox
Service	Python 3.10+ — <i>server_classcord.py</i> — port TCP/12345
Démarrage auto	systemd — utilisateur dédié classcord
Conteneurisation	Docker + docker-compose — image reproductible
Versioning	Git + GitHub — tags de version (v1.0...)
Sécurité	ufw / iptables — fail2ban — filtrage IP
Persistance	SQLite (users, messages) — sauvegarde users.pkl via cron
Exploitation	Logs applicatifs/système — journalctl — cron backups

3. Architecture réseau — NAT + Port Forwarding

VM Linux	Debian/Ubuntu — service ClassCord en écoute sur TCP/12345
Accès client	Client SLAM connecté à l'IP de l'hôte physique → redirigé vers la VM (NAT)
Port forwarding	hôte:12345 → vm:12345 (TCP) — point central pour interopérabilité pédagogique
Réseau	VM en NAT — accessible depuis le réseau pédagogique cloisonné via l'hôte

ClassCord Server — Service de tchat multi-clients

Python 3.10+ · systemd · Docker · Sécurité · Interopérabilité SLAM — Semaine intensive

4. Plan de réalisation — Semaine intensive

Jour	Activités
Jour 1 — Mise en route	Clonage dépôt, lecture server_classcord.py, lancement local, test port 12345, début README
Jour 2 — Réseau + Service	Accessibilité SLAM (NAT/port forwarding), service systemd, 1ère image Docker, DOC_CONNEXION.md
Jour 3 — Sécurité + Backups	Logging structuré, fail2ban, filtrage accès, sauvegarde périodique users.pkl, SECURITE.md
Jour 4 — Évolutions fonct.	Canaux de discussion, persistance SQLite, messages système, logs enrichis, FONCTIONNALITES.md
Jour 5 — Finalisation	Tests croisés interopérabilité (≥2 clients), nettoyage dépôt, validation Docker, fiche RP

Commandes clés d'exploitation

```
# Démarrer le service ClassCord
sudo systemctl start classcord
sudo systemctl enable classcord

# Vérifier port en écoute
ss -tlnp | grep 12345

# Logs en temps réel
journalctl -u classcord -f

# Lancement via Docker
docker compose up -d && docker compose ps
```

6. Risques et recommandations

Risque identifié	Recommandation
Confusion IP VM / IP hôte lors des tests externes	Documenter clairement l'IP hôte + port forwarding dans DOC_CONNEXION.md
Absence de persistance robuste des données	Basculer sur SQLite dès que possible — sauvegardes cron quotidiennes
Documentation incomplète pour reprise par un tiers	README à jour + procédures courtes de dépannage (service down, port fermé)
Service non redémarré après reboot	systemctl enable classcord — vérifier au démarrage VM

7. Livrables et compétences mobilisées

Livrables		Compétences BTS SIO	
Serveur	Service ClassCord accessible, stable, documenté	B1.5	Mettre à disposition un service informatique
Scripts	systemd + cron (sauvegardes) opérationnels	B3.1	Concevoir et mettre en place une solution réseau
Sécurité	ufw, fail2ban, filtrage IP — SECURITE.md	B3.2	Exploiter, dépanner et superviser une solution réseau
Docker	Image conteneurisée fonctionnelle + docker-compose		
Dépôt Git	README + annexes — tags v1.0 pour maintenance		